

Architectural Synthesis of Hybrid Instruction Set Computing (HISC): Leveraging Heterogeneous Pipelines for Energy-Efficient Edge Computing

Castor *et al.*

Advanced Computing Research Initiative

March 2026

Abstract

This paper introduces and analyzes the Hybrid Instruction Set Computing (HISC) architecture, a novel processor design that dynamically routes instructions between RISC and CISC processing units[cite: 2]. By utilizing a specialized split L1 cache hierarchy and a robust Hardware Abstraction Layer (HAL), HISC optimizes for both high-frequency atomic operations and complex macro-instructions[cite: 3, 185]. We explore the transition of HISC from high-performance computing to power-constrained environments such as mobile and IoT, demonstrating its potential to coexist with future quantum paradigms[cite: 140, 163].

1 Introduction

The evolution of processor design has historically vacillated between the streamlined efficiency of Reduced Instruction Set Computing (RISC) and the semantic density of Complex Instruction Set Computing (CISC)[cite: 1, 189]. Modern computational demands, particularly in AI and real-time data analysis, require a synthesis of these philosophies[cite: 6]. HISC proposes a unified silicon footprint where instructions are intelligently routed to the optimal pipeline based on computational complexity[cite: 2, 17].

2 Literature Review

The philosophical precursors to HISC include ARM's *big.LITTLE* architecture, which employs power-efficient and high-performance cores [cite: 15], and heterogeneous computing models that pair CPUs with GPUs[cite: 16]. However, HISC diverges by integrating disparate instruction set architectures (ISAs) into a single, dynamically scheduled execution environment[cite: 17, 31].

3 Hypotheses

- **H1:** A split L1 cache architecture reduces inter-pipeline interference and allows for ISA-specific latency tuning[cite: 79, 120].
- **H2:** A Hardware Abstraction Layer (HAL) effectively masks ISA complexity, ensuring software portability across the hybrid core[cite: 170, 186].
- **H3:** HISC provides superior power-per-watt metrics in mobile SoCs compared to monolithic ISA designs[cite: 142].

4 Methodology Architectural Design

4.1 Microarchitecture and Pipeline Routing

The HISC core employs an *Instruction Profiler* to monitor sequences at the firmware level[cite: 32]. Simple arithmetic operations are shuttled to a short RISC pipeline, while complex string manipulations or specialized DSP tasks are routed to a microcoded CISC pipeline[cite: 35, 109].

4.2 Memory and Cache Hierarchy

To prevent "toe-stepping" between pipelines, we implement:

- **Split L1 Caches:** The RISC L1 utilizes 32-byte lines for low-latency, high-frequency fetches, while the CISC L1 utilizes 64-byte lines for instruction density[cite: 89, 91].
- **Unified L2/L3:** A centralized coherence engine manages shared data, facilitating seamless collaboration between pipelines without L1 thrashing[cite: 82, 114].

4.3 Software Stack (HAL)

The HAL serves as the critical intermediary, defining generic APIs for system operations[cite: 177]. This allows the OS to see a uniform execution target while the hardware manages the underlying dispatch logic[cite: 49, 186].

5 Results and Outcomes

Initial theoretical modeling suggests that:

- **Efficiency:** The split L1 design provides optimized latency for distinct fetch patterns, maintaining high IPC for RISC workloads[cite: 108, 119].
- **Flexibility:** Compilers (e.g., LLVM-based) can generate "fat" binaries containing routines optimized for both pipelines, selected at runtime based on power state[cite: 44, 46].
- **Market Viability:** In the IoT and mobile sectors, HISC offers a unique niche by providing a "brawny" pipeline for bursty encryption or AI tasks alongside a "lean" pipeline for background polling[cite: 150, 156].

6 Conclusion Further Work

HISC represents a significant leap in classical architecture, particularly relevant for edge devices where quantum computing remains impractical[cite: 137, 164]. While manufacturing complexity remains a challenge, the potential for power-efficient, specialized computing is immense[cite: 7, 142].

Further Work:

1. **Software Simulation:** Extending QEMU or gem5 to model dual-pipeline dispatch[cite: 56].
2. **FPGA Prototyping:** Validating the HAL and split-cache coherence in Verilog/VHDL[cite: 58, 59].
3. **Ecosystem Development:** Creating a Linux-based kernel with HISC-aware drivers[cite: 63, 64].

Final Thoughts

The transition from theoretical HISC to physical silicon requires an iterative approach—starting with simulation and moving toward ASIC tape-out[cite: 55, 60]. By focusing on the mobile and embedded markets, HISC can carve out a long-term coexistence strategy with emerging high-level computing paradigms[cite: 163, 165].